3008ICT Deep Learning

Trimester 1, 2026

Assignment 2: Recurrent Neural Networks and NLP

• Due: Monday 18th May 2026, 11:59 PM.

• Total Marks: 30%:

Instructions:

• Late submission: Late submission is allowed but a penalty applies. The penalty is defined as the reduction of the mark allocated to the assessment item by 5% of the total weighted mark for the assessment item, for each working day that the item is late. A working day is defined as Monday to Friday. Assessment items submitted more than five working days after the due date will be awarded zero marks.

• Extension: You can request an extension of time on one of two grounds: medical or special circumstances (e.g. special family or personal circumstances, unavoidable commitments). Please note that workloads such as other assignments are not eligible grounds for extensions. All requests must be made through the myGriffith portal.

• Individual Work: You must complete this assignment individually and submit authentic work. This should be your own work. Anyone caught plagiarising will be penalised and reported to the university. Students are not allowed to use AI to generate solutions for the assignment (code or report). You must demonstrate your work in the submission.

Objectives:

✓ Implement and compare three different types of recurrent neural networks (RNNs).

✓ Classify movie reviews as positive or negative (sentiment analysis) using the IMDb dataset.

✓ Evaluate the performance of the implementations on a real-world dataset.

Your code should be written in PyTorch (Python). In rare cases, students may be allowed to use another programming language, but approval by the course convenor is required.

This assignment can be done on a CPU, while students are encouraged to use GPU if available.

No heavy math is required: you will work at the layer level, letting PyTorch handle the internals.

Prerequisites: Python 3.9+, PyTorch 2.x, torchtext or datasets (HuggingFace), numpy, matplotlib.

Install with: pip install torch torchtext datasets

You can train on a subset of the dataset first, say 5 000-samples, to verify your code runs, then use the full dataset for final results.

Submission:

1. A single Jupyter notebook rnn_sentiment.ipynb with all cells executed.
2. Short PDF report (expected No More than 2 pages, but no page limit).

Part 0. Setup & Data Preprocessing

All three models share the same data pipeline. Complete this section once; it will be reused in every subsequent part.

0.1. Load the IMDb Dataset

Use the HuggingFace datasets library to load IMDb. The dataset has 25 000 training reviews and 25 000 test reviews, each labeled 0 (negative) or 1 (positive).

In [1]:
```
from datasets import load_dataset

dataset = load_dataset("imdb")
train_data = dataset["train"]
test_data  = dataset["test"]

# Quick look
print(train_data[0]["text"][:200])
print("Label:", train_data[0]["label"])
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set
a HF_TOKEN to enable higher rate limits and faster downloads.
I rented I AM CURIOUS-YELLOW from my video store because of all the controve
rsy that surrounded it when it was first released in 1967. I also heard that
at first it was seized by U.S. customs if it ev
Label: 0
```

0.2. Tokenisation & Vocabulary

Convert raw text into integer token sequences. We keep the vocabulary to the 20 000 most common words and cap each review at 500 tokens.

In [7]:
```
import re
from collections import Counter
import torch
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence

# ── helpers ──────────────────────────────────────────────
def tokenize(text):
    text = text.lower()
    return re.findall(r"\b[a-z']+\b", text)

# Build vocab from training set
VOCAB_SIZE = 20_000
MAX_LEN    = 500
PAD_IDX    = 0
UNK_IDX    = 1

counter = Counter()
for ex in train_data:
    counter.update(tokenize(ex["text"]))

vocab = {"<pad>": PAD_IDX, "<unk>": UNK_IDX}
vocab.update({w: i+2 for i, (w, _) in
              enumerate(counter.most_common(VOCAB_SIZE))})

def encode(text):
    tokens = tokenize(text)[:MAX_LEN]
    return [vocab.get(t, UNK_IDX) for t in tokens]

# ── Dataset class ────────────────────────────────────────
class IMDbDataset(Dataset):
    def __init__(self, split):
        self.samples = [
            (
                torch.tensor(encode(ex["text"]), dtype=torch.long),
                torch.tensor(ex["label"],        dtype=torch.long)
            )
            for ex in dataset[split]
        ]

    def __len__(self): return len(self.samples)
    def __getitem__(self, i): return self.samples[i]


def collate(batch):
    texts, labels = zip(*batch)
    # Capture true lengths BEFORE padding
    lengths = torch.tensor([len(t) for t in texts], dtype=torch.long)
    texts = pad_sequence(texts, batch_first=True, padding_value=PAD_IDX)
    return texts, torch.stack(labels), lengths  # Return lengths
```

```
train_loader = DataLoader(IMDbDataset("train"), batch_size=64,
                          shuffle=True,  collate_fn=collate)
test_loader  = DataLoader(IMDbDataset("test"),  batch_size=64,
                          shuffle=False, collate_fn=collate)
```

Hint: PyTorch's pad_sequence automatically pads shorter reviews with zeros so all sequences in a batch have the same length. This is necessary for batched processing.

PART 1. Vanilla RNN Sentiment Classifier

A vanilla RNN processes text one token at a time, left to right, maintaining a hidden state that acts as its "memory". Implement the model below and complete the training loop.

The model architecture has three stages: Embed → Recur → Classify.

TASK 1.1. DEFINE BASIC RNN MODEL and TRAINING LOOP

Complete the function train_model(model, loader, optimizer, criterion, device) that iterates over one epoch and returns the average loss and accuracy. Then write a corresponding evaluate_model function for the test set (no gradient computation needed).

REQUIREMENTS

Use torch.optim.Adam with lr=1e-3

Use nn.CrossEntropyLoss

Train for 5 epochs and print loss & accuracy each epoch

Move model and data to device (use GPU if available)

In [3]:
```
import torch.nn as nn

class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_classes, dropout
        super().__init__()
        """ Your Code"""

    def forward(self, x):
        """ Your Code"""

# ── Shared training utilities ──────────────────────────────────
def train_model(model, loader, optimizer, criterion, device):
    """One training epoch. Returns (avg_loss, accuracy)."""

    """ Your Code """


def evaluate_model(model, loader, criterion, device):
```

```
    """Evaluation pass (no gradients). Returns (avg_loss, accuracy)."""

    """ Your Code """

def run_training(model, train_loader, test_loader, n_epochs=5, lr=1e-3):
    """Full training run. Returns history dict."""

    """ Your Code """


# Initiate and Train
model_rnn = RNNClassifier(
    vocab_size = len(vocab),
    embed_dim  = 128,
    hidden_dim = 256,
    n_classes  = 2
)
print(model_rnn)
```

```
RNNClassifier()
```

Hint: Use device = torch.device("cuda" if torch.cuda.is_available() else "cpu") and call model.to(device) once before training.

TASK 1.2. OBSERVATIONS

In a markdown cell, answer the following:

(1). What test accuracy did your vanilla RNN achieve?

(2). Plot training loss vs. epoch. Does the model converge? Any sign of overfitting?

(3). What is the vanishing gradient problem and why might it affect this model on long reviews? (2–3 sentences, no equations needed.)

PART 2. LSTM Sentiment Classifier: From RNN to LSTM

The Long Short-Term Memory (LSTM) adds gates that let the model decide what to remember and what to forget. In PyTorch, swapping an RNN for an LSTM requires only one line change — but the performance gain is substantial.

The key difference: nn.LSTM returns (output, (hidden, cell)) instead of (output, hidden). We use only the final hidden state for classification.

Task 2.1. DEFINE LSTM MODEL

In [4]:
```
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_classes,
```

```
                 n_layers=2, dropout=0.5):
        """ Your Code"""

    def forward(self, x):
        """ Your Code"""

model_lstm = LSTMClassifier(
    vocab_size = len(vocab),
    embed_dim  = 128,
    hidden_dim = 256,
    n_classes  = 2,
    n_layers   = 2
)
```

TASK 2.2. TRAIN THE LSTM

Re-use your training loop from Part 1 to train model_lstm for 5 epochs with the same hyperparameters.

REQUIREMENTS

(1). Report final test accuracy.

(2). On the same plot, show training loss curves for both the RNN and the LSTM. Label each line clearly.

(3). Which model trains faster (loss drops quicker)? Why might that be?

PART 3. Bidirectional LSTM

A unidirectional LSTM only reads text left-to-right. A Bidirectional LSTM (BRNN) runs two LSTMs in parallel — one forward, one backward — and concatenates their hidden states. This gives every token access to both its past and its future context.

Note: Because the two directions are concatenated, the final hidden dimension is 2 × hidden_dim. Adjust your classifier head accordingly.

Only two changes from LSTMClassifier: add bidirectional=True to nn.LSTM, and double the input size of nn.Linear.

TASK 3.1. DEFINE BRNN (B-LSTM)

In [5]:
```
class BRNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_classes,
                 n_layers=2, dropout=0.5):
        """ Your Code"""

    def forward(self, x):
        """ Your Code"""
```

```
model_brnn = BRNNClassifier(
    vocab_size = len(vocab),
    embed_dim  = 128,
    hidden_dim = 256,
    n_classes  = 2,
    n_layers   = 2
)
```

TASK 3.2. TRAIN THE BRNN

Re-use your training loop from Part 1 to train model_brnn for 5 epochs with the same hyperparameters.

REQUIREMENTS

(1). Report final test accuracy.

(2). On the same plot, show training loss curves for the RNN, the LSTM and the BRNN. Label each line clearly.

(3). Which model trains faster (loss drops quicker)? Why might that be?

PART 4. Experiments, Analysis and Report

TASK 4.1. HEAD-TO-HEAD SUMMARY TABLE

Fill in the following table in your notebook. All values should come from your actual runs.

Model	TestAccuracy	Parameters	Avg Epoch Time
Vanilla RNN	---	---	---
LSTM	---	---	---
BRNN	---	---	---

Write 3–4 sentences interpreting the trade-off between accuracy, model size, and training speed across the three architectures.

TASK 4.2. LIVE PREDICTION FUNCTION

Implement predict_sentiment(text, model, vocab, device) that takes a raw English string and returns "Positive" or "Negative" along with the confidence score. Test it on at least four reviews of your own invention (two obviously positive, two ambiguous).

In [6]:
```
# Your Code
```

TASK 4.3. ERROR ANALYSIS

Find 5 misclassified examples from the test set using your best model. For each, print the first 100 tokens of the review, the true label, and the predicted label.

Identify at least two patterns that explain the errors. Common culprits: sarcasm, negation ("not bad"), domain-specific words, very long reviews.

Hint: Iterate over test_loader, collect predictions, and compare to true labels. You can use torch.where(preds != labels) to find mismatches.

Note: Code quality matters. Use clear variable names and add a comment to every non-obvious line. Submissions with no comments will lose up to 10 points.

TIPS

Gradient clipping: Add torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) before optimizer.step() to stabilise RNN training.

Device consistency: If you get a device mismatch error, ensure both model and data tensors are on the same device.

Hidden state shape: For a 2-layer bidirectional LSTM, hidden has shape (4, batch, hidden_dim) — 2 layers × 2 directions. Indices -2 and -1 are the last layer's forward and backward states.

Reproducibility: Set seeds at the top of your notebook: torch.manual_seed(42), import random; random.seed(42).

Slow training? You can train on a 5 000-sample subset first to verify your code runs, then use the full dataset for final results.

Task 4.4. Report

You should write a report for the above tasks. You might wish to include a few necessary screenshots in the report.

The file for the document must be in one of the two formats: Microsoft Word (.doc/.docx), or Adobe PDF (.pdf).

Your report should be no more than 2 pages. In case you do need more pages, there is no penalty. But basic criteria of scientific writing should be followed.

a. Softwaredesign: a brief description of the software and your code.

b. The test results and diagrams (tables/curves).

c. Discuss the results, include any observations or insights you gained from running tests.

d. Your feedback on the assginment (optional)